In [ ]:
```
#slip 1
```

In [ ]:
```
Q1.  Python program that demonstrates the hill climbing algorithm to find the maximum of a
mathematical function.(For example f(x) = -x^2 + 4x
```

In [15]:
```
def objective_function(x):
    return -x**2 + 4*x
def hill_climbing(initial_x, step_size, num_steps):
    current_x = initial_x
    for step in range(num_steps):
        current_value = objective_function(current_x)
        next_x = current_x + step_size
        next_value = objective_function(next_x)
        if next_value > current_value:
            current_x = next_x
    return current_x, objective_function(current_x)
initial_x = 0.0
step_size = 0.1
num_steps = 100
result_x, result_value = hill_climbing(initial_x, step_size, num_steps)
print(f"Maximum found at x = {result_x}")
print(f"Maximum value is {result_value}")
```

```
Maximum found at x = 2.0000000000000004
Maximum value is 4.0
```

In [ ]:
```
 Q.2) Write a Python program to implement Depth First Search algorithm.
 Refer the following graph as an Input for the program. [Initial node=1,Goal node=8].
```

In [16]:
```
graph = {
  '1' : ['3','2'],
  '3' : ['2'],
  '2' : ['4'],
  '5' : ['3','7'],
  '4' : ['5','6'],
  '7' : ['6'],
  '6' :[]}
visited = set()
def dfs(visited, graph, node):
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
print("Following is the Depth-First Search")
dfs(visited, graph, '1')
```

```
Following is the Depth-First Search
1
3
2
4
5
7
6
```

In [ ]:
```
#slip 2
```

In [ ]:
```
Q1. Write a python program to generate Calendar for the given month and year?
```

In [14]:
```
import calendar
def generate_month_calendar():
    try:
        year = int(input("Enter the year (e.g., 2024): "))
        month = int(input("Enter the month (1-12): "))
        if not (1 <= month <= 12):
            print("Invalid month! Please enter a number between 1 and 12.")
            return
        print(f"\nCalendar for {calendar.month_name[month]} {year}:\n")
        print(calendar.month(year, month))
    except ValueError:
        print("Invalid input! Please enter valid numeric values for year and month.")
if __name__ == "__main__":
    generate_month_calendar()
```

```
Calendar for September 2025:

   September 2025
Mo Tu We Th Fr Sa Su
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14

15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30
```

In [ ]:
```
Q2. Write a Python program to implement Depth First Search algorithm.
    Refer the following graph as an Input for the program.[Initial node=1,Goal node=7].
```

In [ ]:
```
#slip1 (Q2)
```

In [ ]:
```
#slip3
```

In [ ]:
```
Q1. Write a python program to remove punctuations from the given string?
```

In [13]:
```
import string
def remove_punctuations(text):
    translator = str.maketrans('', '', string.punctuation)
    cleaned_text = text.translate(translator)
    return cleaned_text
input_string = "Hello! How are you? I'm doing well, thanks."
cleaned_string = remove_punctuations(input_string)
print(f"Original string: {input_string}")
print(f"String after removing punctuations: {cleaned_string}")
```

```
Original string: Hello! How are you? I'm doing well, thanks.
String after removing punctuations: Hello How are you Im doing well thanks
```

In [ ]:
```
Q2.  Write a Python program to implement Depth First Search algorithm.
    Refer the following graph as an Input for the program.[Initial node=2,Goal node=7]
```

In [ ]:
```
#slip1 (Q2)
```

In [ ]:
```
#slip4
```

In [ ]:
```
Q1.Write a program to implement different Set Operations.(any 5 operations)
```

In [17]:
```
set1 = {1, 2, 3, 4, 5}
set2 = {3, 4, 5, 6, 7}
union_set = set1 | set2
intersection_set = set1 & set2
difference_set = set1 - set2
symmetric_difference_set = set1 ^ set2
print("Set 1:", set1)
print("Set 2:", set2)
print("Union Set:", union_set)
print("Intersection Set:", intersection_set)
print("Difference Set:", difference_set)
print("Symmetric Difference Set:", symmetric_difference_set)
```

```
Set 1: {1, 2, 3, 4, 5}
Set 2: {3, 4, 5, 6, 7}
Union Set: {1, 2, 3, 4, 5, 6, 7}
Intersection Set: {3, 4, 5}
Difference Set: {1, 2}
Symmetric Difference Set: {1, 2, 6, 7}
```

In [ ]:
```
Q2.Write a Python program to implement Breadth First Search algorithm.
    Refer the following graph as an Input for the program.[Initial node=1,Goal node=8]
```

In [18]:
```
graph = {
  '5' : ['3','7'],
  '3' : ['2', '4'],
  '7' : ['8'],
  '2' : [],
  '4' : ['8'],
  '8' : []}
visited = []
queue = []
def bfs(visited, graph, node):
  visited.append(node)
  queue.append(node)
  while queue:
    m = queue.pop(0)
    print (m, end = " ")
    for neighbour in graph[m]:
```

```
      if neighbour not in visited:
        visited.append(neighbour)
        queue.append(neighbour)
print("Following is the Breadth-First Search")
bfs(visited, graph, '5')
```

```
Following is the Breadth-First Search
5 3 7 2 4 8
```

In [ ]:
```
#slip5
```

In [ ]:
```
Q1. Write a python program to implement List operations(any 5 operations)
```

In [19]:
```
my_list = [10, 20, 30, 40, 50]
print(f"Original list: {my_list}")
my_list.append(60)
print(f"After append(60): {my_list}")
my_list.insert(1, 15)
print(f"After insert(1, 15): {my_list}")
my_list.remove(30)
print(f"After remove(30): {my_list}")
popped_element = my_list.pop(2)
print(f"After pop(2): {my_list}")
print(f"Popped element: {popped_element}")
my_list.sort()
print(f"After sort(): {my_list}")
```

```
Original list: [10, 20, 30, 40, 50]
After append(60): [10, 20, 30, 40, 50, 60]
After insert(1, 15): [10, 15, 20, 30, 40, 50, 60]
After remove(30): [10, 15, 20, 40, 50, 60]
After pop(2): [10, 15, 40, 50, 60]
Popped element: 20
After sort(): [10, 15, 40, 50, 60]
```

In [ ]:
```
Q2. Write a Python program to implement Breadth First Search algorithm.
    Refer the following graph as an Input for the program.[Initial node=1,Goal node=8]
```

In [ ]:
```
#slip4 (Q2)
```

In [ ]:
```
#slip6
```

In [ ]:
```
Q1. Write a python program to print lowercase and uppercase alphabets of string
```

In [20]:
```
def print_case_alphabets(input_string):
    lowercase_alphabets = []
    uppercase_alphabets = []
    for char in input_string:
        if 'a' <= char <= 'z':
            lowercase_alphabets.append(char)
        elif 'A' <= char <= 'Z':
            uppercase_alphabets.append(char)
    print(f"Lowercase alphabets: {''.join(lowercase_alphabets)}")
    print(f"Uppercase alphabets: {''.join(uppercase_alphabets)}")
my_string = "Hello World 123 Python"
print_case_alphabets(my_string)
```

```
Lowercase alphabets: elloorldython
Uppercase alphabets: HWP
```

In [ ]:
```
Q2. Write a Python program to implement Breadth First Search algorithm.
    Refer the following graph as an Input for the program.[Initial node=1,Goal node=8].
```

In [ ]:
```
#slip4 (Q2)
```

In [ ]:
```
#slip7
```

In [ ]:
```
Q1. Write a python program to accept a number and check whether  prime or not
```

In [21]:
```
def is_prime(number):
    if number <= 1:
        return False
    for i in range(2, int(number**0.5) + 1):
        if number % i == 0:
            return False
    return True
try:
    num = int(input("Enter a number: "))
    if is_prime(num):
        print(f"{num} is a prime number.")
    else:
        print(f"{num} is not a prime number.")
```

```
except ValueError:
    print("Invalid input. Please enter an integer.")
```

```
5 is a prime number.
```

In [ ]:
```
Q2. Write a Python program to solve 4 x 4 Queen  problem .
```

In [25]:
```
def solve_n_queens(n):
    board = [['.' for _ in range(n)] for _ in range(n)]
    solutions = []
    def is_safe(row, col):
        for c in range(col):
            if board[row][c] == 'Q':
                return False
        r, c = row, col
        while r >= 0 and c >= 0:
            if board[r][c] == 'Q':
                return False
            r -= 1
            c -= 1
        r, c = row, col
        while r < n and c >= 0:
            if board[r][c] == 'Q':
                return False
            r += 1
            c -= 1
        return True
    def backtrack(col):
        if col >= n:
            current_solution = ["".join(row) for row in board]
            solutions.append(current_solution)
            return
        for row in range(n):
            if is_safe(row, col):
                board[row][col] = 'Q'
                backtrack(col + 1)
                board[row][col] = '.'
    backtrack(0)
    return solutions
def print_solutions(solutions):
    if not solutions:
        print("No solutions found.")
        return
    for i, solution in enumerate(solutions):
        print(f"Solution {i+1}:")
        for row_str in solution:
            print(row_str)
        print()
n = 4
found_solutions = solve_n_queens(n)
print_solutions(found_solutions)
```

```
Solution 1:
..Q.
Q...
...Q
.Q..

Solution 2:
.Q..
...Q
Q...
..Q.
```

In [ ]:
```
#slip8
```

In [ ]:
```
 Q1.Write a Python program to accept a string.
     Find and print the number of upper case alphabets and lower case alphabets.
```

In [4]:
```
def count_case_alphabets(input_string):
    uppercase_count = 0
    lowercase_count = 0

    for char in input_string:
        if char.isupper():
            uppercase_count += 1
        elif char.islower():
            lowercase_count += 1

    return uppercase_count, lowercase_count
user_string = input("Enter a string: ")
upper, lower = count_case_alphabets(user_string)
```

```
print(f"Number of uppercase alphabets: {upper}")
print(f"Number of lowercase alphabets: {lower}")
```

```
Number of uppercase alphabets: 1
Number of lowercase alphabets: 4
```

In [ ]:
```
Q2.Write a Python program to solve N X N Queen problem.
```

In [ ]:
```
#slip7(Q2)
```

In [ ]:
```
#slip9
```

In [ ]:
```
Q1.Write python program to solve 8 puzzle problem using A* algorithm
```

In [7]:
```
import heapq
class PuzzleState:
    def __init__(self, board, parent=None, move=None, g_score=0):
        self.board = board
        self.parent = parent
        self.move = move
        self.g_score = g_score
    def __lt__(self, other):
        return (self.g_score + self.heuristic()) < (other.g_score + other.heuristic())
    def __eq__(self, other):
        return self.board == other.board
    def __hash__(self):
        return hash(self.board)
    def heuristic(self):
        distance = 0
        goal_state = (1, 2, 3, 4, 5, 6, 7, 8, 0)
        for i in range(9):
            if self.board[i] != 0:
                x1, y1 = divmod(i, 3)
                x2, y2 = divmod(goal_state.index(self.board[i]), 3)
                distance += abs(x1 - x2) + abs(y1 - y2)
        return distance
    def get_blank_position(self):
        return self.board.index(0)
    def get_neighbors(self):
        neighbors = []
        blank_idx = self.get_blank_position()
        blank_row, blank_col = divmod(blank_idx, 3)
        moves = [(-1, 0, 'U'), (1, 0, 'D'), (0, -1, 'L'), (0, 1, 'R')]
        for dr, dc,move_char in moves:
            new_row, new_col = blank_row + dr, blank_col + dc
            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_blank_idx = new_row * 3 + new_col
                new_board_list = list(self.board)
                new_board_list[blank_idx], new_board_list[new_blank_idx] = \
                    new_board_list[new_blank_idx], new_board_list[blank_idx]
                neighbors.append(PuzzleState(tuple(new_board_list), self, move_char, self.g_score + 1))
        return neighbors
def solve_8_puzzle(initial_board):
    goal_board = (1, 2, 3, 4, 5, 6, 7, 8, 0)
    initial_state = PuzzleState(tuple(initial_board))
    open_list = []
    heapq.heappush(open_list, initial_state)
    closed_set = set()
    while open_list:
        current_state = heapq.heappop(open_list)
        if current_state.board == goal_board:
            path = []
            while current_state:
                path.append(current_state.board)
                current_state = current_state.parent
            return path[::-1]
        if current_state.board in closed_set:
            continue
        closed_set.add(current_state.board)
        for neighbor in current_state.get_neighbors():
            if neighbor.board not in closed_set:
                heapq.heappush(open_list, neighbor)
    return None
def print_solution(path):
    if path:
        for i, state in enumerate(path):
            print(f"Move {i}:")
            for r in range(0, 9, 3):
                print(f"{state[r]} {state[r+1]} {state[r+2]}")
            print("-" * 10)
    else:
        print("No solution found.")
initial_board = [1, 2, 3, 4, 0, 5, 7, 8, 6]
```

```
solution_path = solve_8_puzzle(initial_board)
print_solution(solution_path)
```

```
Move 0:
1 2 3
4 0 5
7 8 6
----------
Move 1:
1 2 3
4 5 0
7 8 6
----------
Move 2:
1 2 3
4 5 6
7 8 0
----------
```

In [ ]:
```
Q2.Write a Python program to solve water jug problem.
    2 jugs with capacity 5 gallon and 7 gallon are given with unlimited water supply respectively.
    The target to achieve is 4 gallon of water in second jug
```

In [9]:
```
from collections import deque
def solve_water_jug_problem(jug1_capacity, jug2_capacity, target_amount_in_jug2):
    queue = deque([((0, 0), [(0, 0)])])
    visited = set()
    visited.add((0, 0))
    while queue:
        (jug1, jug2), path = queue.popleft()
        if jug2 == target_amount_in_jug2:
            return path
        actions = [
            (jug1_capacity, jug2),
            (jug1, jug2_capacity),
            (0, jug2),
            (jug1, 0),
            (jug1 - min(jug1, jug2_capacity - jug2), jug2 + min(jug1, jug2_capacity - jug2)),
            (jug1 + min(jug2, jug1_capacity - jug1), jug2 - min(jug2, jug1_capacity - jug1))]
        for next_jug1, next_jug2 in actions:
            if (next_jug1, next_jug2) not in visited:
                visited.add((next_jug1, next_jug2))
                queue.append(((next_jug1, next_jug2), path + [(next_jug1, next_jug2)]))
    return None
jug1_capacity = 5
jug2_capacity = 7
target = 4
solution_path = solve_water_jug_problem(jug1_capacity, jug2_capacity, target)
if solution_path:
    print("Path to achieve 4 gallons in the 7-gallon jug:")
    for state in solution_path:
        print(f"Jug 1: {state[0]} gallons, Jug 2: {state[1]} gallons")
else:
    print("No solution found for the given parameters.")
```

```
Path to achieve 4 gallons in the 7-gallon jug:
Jug 1: 0 gallons, Jug 2: 0 gallons
Jug 1: 0 gallons, Jug 2: 7 gallons
Jug 1: 5 gallons, Jug 2: 2 gallons
Jug 1: 0 gallons, Jug 2: 2 gallons
Jug 1: 2 gallons, Jug 2: 0 gallons
Jug 1: 2 gallons, Jug 2: 7 gallons
Jug 1: 5 gallons, Jug 2: 4 gallons
```

In [ ]:
```
#slip10
```

In [ ]:
```
Q1.  Write Python program to implement crypt arithmetic problem
TWO+TWO=FOUR
```

In [11]:
```
import itertools
def solve_cryptarithmetic():
    letters = ('T', 'W', 'O', 'F', 'U', 'R')
    for perm in itertools.permutations(range(10), len(letters)):
        mapping = dict(zip(letters, perm))
        if mapping['T'] == 0 or mapping['F'] == 0:
            continue
        TWO = mapping['T'] * 100 + mapping['W'] * 10 + mapping['O']
        FOUR = mapping['F'] * 1000 + mapping['O'] * 100 + mapping['U'] * 10 + mapping['R']
        if TWO + TWO == FOUR:
            print(f"Solution found:")
            print(f"T = {mapping['T']}")
            print(f"W = {mapping['W']}")
            print(f"O = {mapping['O']}")
```

```
            print(f"F = {mapping['F']}")
            print(f"U = {mapping['U']}")
            print(f"R = {mapping['R']}")
            print(f"Result: {TWO} + {TWO} = {FOUR}")
            return True
    print("No solution found.")
    return False
if __name__ == "__main__":
    solve_cryptarithmetic()
```

```
Solution found:
T = 7
W = 3
O = 4
F = 1
U = 6
R = 8
Result: 734 + 734 = 1468
```

In [ ]:
```
Q2.Write a Python program to implement Simple Chatbot.
```

In [28]:
```
responses = {
    "hi": "Hello there! How can I help you today?",
    "hello": "Hi! How can I assist you?",
    "hey": "Hey! What can I do for you?",
    "how are you": "I'm just a computer program, but I'm here to help you.",
    "bye": "Goodbye! Have a great day.",
    "exit": "Goodbye! If you have more questions, feel free to come back."
}
def chatbot(user_input):
    user_input = user_input.lower()
    response = responses.get(user_input, "I'm not sure how to respond to that. Please choose from the predefine
    return response
print("Simple Chatbot: Type 'bye' to exit")
while True:
    user_input = input("You: ")
    if user_input.lower() == "bye" or user_input.lower() == "exit":
        print("Simple Chatbot: Goodbye!")
        break
    response = chatbot(user_input)
    print("Simple Chatbot:", response)
```

```
Simple Chatbot: Type 'bye' to exit
Simple Chatbot: Hello there! How can I help you today?
Simple Chatbot: Goodbye!
```

In [ ]:
```
#slip11
```

In [ ]:
```
Q1. Write a python program using mean end analysis algorithm problem of transforming
a string of lowercase letters into another string.
```

In [43]:
```
def means_end_analysis(start: str, goal: str):
    steps = []
    current = list(start)
    goal = list(goal)
    i = 0
    while i < len(current) or i < len(goal):
        if i < len(current) and i < len(goal):
            if current[i] != goal[i]:
                steps.append(f"Replace '{current[i]}' with '{goal[i]}' at position {i}")
                current[i] = goal[i]
            i += 1
        elif i >= len(current):
            current.append(goal[i])
            steps.append(f"Insert '{goal[i]}' at position {i}")
            i += 1
        elif i >= len(goal):
            removed_char = current.pop(i)
            steps.append(f"Delete '{removed_char}' from position {i}")
    return steps, ''.join(current)
if __name__ == "__main__":
    start_string = "cat"
    goal_string = "doge"
    steps, result = means_end_analysis(start_string, goal_string)
    print(f"Start: {start_string}")
    print(f"Goal : {goal_string}")
    print("\nSteps:")
    for step in steps:
        print(" -", step)
    print(f"\nFinal result: {result}")
```

```
Start: cat
Goal : doge

Steps:
 - Replace 'c' with 'd' at position 0
 - Replace 'a' with 'o' at position 1
 - Replace 't' with 'g' at position 2
 - Insert 'e' at position 3

Final result: doge
```

In [ ]:
```
Q2.Write a Python program to solve water jug problem.
    Two jugs with capacity 4 gallon and 3 gallon are given with unlimited water supply respectively.
    The target is to achieve 2 gallon of water in second jug
```

In [ ]:
```
#slip9(Q2)
```

In [ ]:
```
#slip12
```

In [ ]:
```
Q1.Write a python program to generate Calendar for the given month and year?.
```

In [ ]:
```
#slip2(Q1)
```

In [ ]:
```
Q2.Write a Python program to simulate 4-Queens problem
```

In [ ]:
```
#slip7(Q2)
```

In [ ]:
```
#slip13
```

In [ ]:
```
Q1.Write a Python program to implement Mini-Max Algorithm.
```

In [50]:
```
import math
def minimax (curDepth, nodeIndex,
maxTurn, scores,
targetDepth):
  if (curDepth == targetDepth):
      return scores[nodeIndex]
  if (maxTurn):
      return max(minimax(curDepth + 1, nodeIndex * 2,
      False, scores, targetDepth),
      minimax(curDepth + 1, nodeIndex * 2 + 1,
      False, scores, targetDepth))
  else:
      return min(minimax(curDepth + 1, nodeIndex * 2,
      True, scores, targetDepth),
      minimax(curDepth + 1, nodeIndex * 2 + 1,
      True, scores, targetDepth))
scores = [3, 5, 2, 9, 12, 5, 23, 23]
treeDepth = math.log(len(scores), 2)
print("The optimal value is : ", end = "")
print(minimax(0, 0, True, scores, treeDepth))
```

```
The optimal value is : 12
```

In [ ]:
```
Q2.Write a Python program to simulate 8-Queens problem
```

In [55]:
```
def print_chessboard(chessboard):
    for row in chessboard:
        print(" ".join(row))
def is_safe(chessboard, row, col, n):
    for i in range(col):
        if chessboard[row][i] == 'Q':
            return False
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if chessboard[i][j] == 'Q':
            return False
    for i, j in zip(range(row, n, 1), range(col, -1, -1)):
        if chessboard[i][j] == 'Q':
            return False
    return True
def solve_eight_queens(chessboard, col, n):
    if col >= n:
        return True
    for i in range(n):
        if is_safe(chessboard, i, col, n):
            chessboard[i][col] = 'Q'
            if solve_eight_queens(chessboard, col + 1, n):
                return True
            chessboard[i][col] = '.'
    return False
def main():
```

```
    n = 8
    chessboard = [['.' for _ in range(n)] for _ in range(n)]
    if solve_eight_queens(chessboard, 0, n):
        print("Solution to the Eight Queens Problem:")
        print_chessboard(chessboard)
    else:
        print("No solution found.")
if __name__ == '__main__':
    main()
```

```
Solution to the Eight Queens Problem:
Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .
```

In [ ]:
```
#slip14
```

In [ ]:
```
Q1.Write a python program to sort the sentence in alphabetical order?
```

In [56]:
```
my_str = "Hello this Is an Example With cased letters"
words = [word.lower() for word in my_str.split()]
words.sort()
print("The sorted words are:")
for word in words:
   print(word)
```

```
The sorted words are:
an
cased
example
hello
is
letters
this
with
```

In [ ]:
```
Q2.Write a Python program to simulate n-Queens problem.
```

In [ ]:
```
#slip13(Q2)
```

In [ ]:
```
#slip15
```

In [ ]:
```
Q1.Write a  python Program to print Fibonacci series
```

In [59]:
```
def fibonacci_series(n_terms):
    if n_terms <= 0:
        print("Please enter a positive integer.")
        return
    elif n_terms == 1:
        print("Fibonacci series:")
        print(0)
        return
    a, b = 0, 1
    print("Fibonacci series:")
    print(a, end=" ")
    if n_terms > 1:
        print(b, end=" ")
    for _ in range(2, n_terms):
        next_term = a + b
        print(next_term, end=" ")
        a = b
        b = next_term
    print()
try:
    num_terms = int(input("Enter the number of terms for the Fibonacci series: "))
    fibonacci_series(num_terms)
except ValueError:
    print("Invalid input. Please enter an integer.")
```

```
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711
```

In [ ]:
```
Q2.Write a program to implement Iterative Deepening DFS algorithm.      [20 Marks ]
[ Goal Node =G]
```

In [ ]:
```
#slip1(Q2)
```

In [ ]:
```
#slip16
```

In [ ]:
```
Q1.Write a Program to Implement Tower of Hanoi using Python
```

In [60]:
```
def tower_of_hanoi(n, source, auxiliary, target):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    tower_of_hanoi(n-1, source, target, auxiliary)
    print(f"Move disk {n} from {source} to {target}")
    tower_of_hanoi(n-1, auxiliary, source, target)
n = 3
tower_of_hanoi(n, 'A', 'B', 'C')
```

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

In [ ]:
```
Q2.Write a Python program to solve tic-tac-toe problem
```

In [ ]:
```
@No solution
```

In [4]:
```
def print_board(board):
    for row in board:
        print(' | '.join(row))
        print('-' * 5)
def check_win(board, player):
    win_conditions = [
        [board[0][0], board[0][1], board[0][2]],
        [board[1][0], board[1][1], board[1][2]],
        [board[2][0], board[2][1], board[2][2]],
        [board[0][0], board[1][0], board[2][0]],
        [board[0][1], board[1][1], board[2][1]],
        [board[0][2], board[1][2], board[2][2]],
        [board[0][0], board[1][1], board[2][2]],
        [board[0][2], board[1][1], board[2][0]]
    ]
    return [player, player, player] in win_conditions
def tic_tac_toe():
    board = [[' ']*3 for _ in range(3)]
    player = 'X'
    for _ in range(9):
        print_board(board)
        row = int(input(f"Player {player}, enter row (0-2): "))
        col = int(input(f"Player {player}, enter col (0-2): "))
        if board[row][col] == ' ':
            board[row][col] = player
        else:
            print("Invalid move, try again.")
            continue
        if check_win(board, player):
            print_board(board)
            print(f"Player {player} wins!")
            return
        player = 'O' if player == 'X' else 'X'
    print_board(board)
    print("It's a draw!")
tic_tac_toe()
```

```
  |   |
-----
  |   |
-----
  |   |
-----
  |   |
-----
  |   | X
-----
  |   |
-----
  | O |
-----
  |   | X
-----
  |   |
-----
```

```
Invalid move, try again.
  | O |
-----
  |   | X
-----
  |   |
-----
  | O | X
-----
  |   | X
-----
  |   |
-----
Invalid move, try again.
  | O | X
-----
  |   | X
-----
  |   |
-----
Invalid move, try again.
  | O | X
-----
  |   | X
-----
  |   |
-----
Invalid move, try again.
  | O | X
-----
  |   | X
-----
  |   |
-----
  | O | X
-----
  |   | X
-----
  |   | O
-----
  | O | X
-----
  |   | X
-----
  | X | O
-----
It's a draw!
```

In [ ]:
```
#slip17
```

In [ ]:
```
Q1.Python program that demonstrates the hill climbing algorithm to find the maximum of a mathematical function.
```

In [2]:
```
def f(x):
    return -(x - 3)**2 + 10
def hill_climbing(start, step=0.1, max_iter=1000):
    current = start
    for _ in range(max_iter):
        neighbors = [current - step, current + step]
        next_move = max(neighbors, key=f)
        if f(next_move) <= f(current):
            break
        current = next_move
    return current, f(current)
max_x, max_val = hill_climbing(start=0)
print(f"Maximum at x = {max_x}, f(x) = {max_val}")
```

```
Maximum at x = 3.0000000000000013, f(x) = 10.0
```

In [ ]:
```
Q2.Write a Python program to implement A* algorithm.
    Refer the following graph as an Input for the program.[ Start vertex is A and Goal Vertex is G]
```

In [5]:
```
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'D': 2, 'E': 5},
    'C': {'F': 3},
    'D': {'G': 6},
    'E': {'G': 2},
    'F': {'G': 1},
    'G': {}
}

heuristic = {
    'A': 7,
```

```
    'B': 6,
    'C': 5,
    'D': 4,
    'E': 3,
    'F': 1,
    'G': 0
}

def a_star(start, goal):
    open_set = {start}
    came_from = {}
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0
    f_score = {node: float('inf') for node in graph}
    f_score[start] = heuristic[start]

    while open_set:
        current = min(open_set, key=lambda n: f_score[n])
        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1]

        open_set.remove(current)
        for neighbor, cost in graph[current].items():
            tentative_g = g_score[current] + cost
            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f_score[neighbor] = tentative_g + heuristic[neighbor]
                open_set.add(neighbor)
    return None

path = a_star('A', 'G')
print("Path found:", path)
```

```
Path found: ['A', 'B', 'E', 'G']
```

In [ ]:
```
#slip18
```

In [ ]:
```
Q1.Write a python program to remove stop words for a given passage from a text file using NLTK?
```

In [ ]:
```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('stopwords')
nltk.download('punkt')
text = "This is a sample sentence showing stopword removal."
stop_words = set(stopwords.words('english'))
tokens = word_tokenize(text.lower())
filtered_tokens = [word for word in tokens if word not in stop_words]
print("Original:", tokens)
print("Filtered:", filtered_tokens)
```

In [ ]:
```
Q2.Implement a system that performs arrangement of some set of objects in a room.
    Assume that you have only 5 rectangular, 4 square-shaped objects.
    Use A* approach for the placement of the objects in room for efficient space utilisation.
    Assume suitable heuristic, and dimensions of objects and rooms. (Informed Search)
```

In [ ]:
```
#No solution
```

In [ ]:
```
#slip19
```

In [ ]:
```
Q1. Write a program to implement Hangman game using python. Description:
Hangman is a classic word-guessing game. The user should guess the word correctly by entering alphabets of the
    The Program will get input as single alphabet from the user and it will matchmaking with the alphabets in t
```

In [2]:
```
import random

def hangman():
    words = ['python', 'hangman', 'challenge', 'programming', 'computer']
    word = random.choice(words)
    guessed = set()
    attempts = 6

    while attempts > 0:
        display = ''.join([ch if ch in guessed else '_' for ch in word])
        print("Word:", display)
        if display == word:
```

```
            print("Congratulations! You guessed the word.")
            break

        guess = input("Enter a single alphabet: ").lower()
        if len(guess) != 1 or not guess.isalpha():
            print("Invalid input. Try again.")
            continue

        if guess in guessed:
            print("You already guessed that letter.")
            continue

        guessed.add(guess)

        if guess not in word:
            attempts -= 1
            print(f"Wrong guess! Attempts left: {attempts}")

    else:
        print(f"Game over! The word was '{word}'.")

hangman()
```

```
Word: _______
Word: _a___a_
Wrong guess! Attempts left: 5
Word: _a___a_
Wrong guess! Attempts left: 4
Word: _a___a_
Wrong guess! Attempts left: 3
Word: _a___a_
Wrong guess! Attempts left: 2
Word: _a___a_
Wrong guess! Attempts left: 1
Word: _a___a_
Word: _a_g_a_
Word: ha_g_a_
Wrong guess! Attempts left: 0
Game over! The word was 'hangman'.
```

In [ ]:
```
Q2.Write a Python program to implement A* algorithm. Refer the following graph as an Input for the program.
```

In [ ]:
```
#slip17(Q2)
```

In [ ]:
```
#slip20
```

In [ ]:
```
Q1.Build a bot which provides all the information related to you in college
```

In [3]:
```
def pvp_college_bot():
    info = {
        "name": "ChatGPT",
        "college": "PVP Senior College",
        "department": "Computer Science",
        "year": "3rd Year",
        "skills": "Python, AI, Machine Learning, Data Science",
        "hobbies": "Coding, Reading Tech Articles, Helping Students",
        "contact": "chatgpt@pvp.edu.in"
    }

    print("Hi! I am your PVP Senior College info bot. Ask me about my details like 'name', 'college', 'departmer

    while True:
        query = input("You: ").lower()
        if query == 'exit':
            print("Bot: Goodbye! Have a great day at PVP Senior College!")
            break
        elif query in info:
            print(f"Bot: {query.capitalize()} - {info[query]}")
        else:
            print("Bot: Sorry, I don't have info on that. Try asking about 'name', 'college', 'department', 'yea

pvp_college_bot()
```

```
Hi! I am your PVP Senior College info bot. Ask me about my details like 'name', 'college', 'department', 'year',
'skills', 'hobbies', or 'contact'. Type 'exit' to quit.
Bot: Sorry, I don't have info on that. Try asking about 'name', 'college', 'department', 'year', 'skills', 'hobb
ies', or 'contact'.
Bot: Name - ChatGPT
Bot: Skills - Python, AI, Machine Learning, Data Science
Bot: College - PVP Senior College
Bot: Year - 3rd Year
Bot: Department - Computer Science
Bot: Hobbies - Coding, Reading Tech Articles, Helping Students
```

```
Bot: Sorry, I don't have info on that. Try asking about 'name', 'college', 'department', 'year', 'skills', 'hobb
ies', or 'contact'.
Bot: Contact - chatgpt@pvp.edu.in
Bot: Goodbye! Have a great day at PVP Senior College!
```

In [ ]:
```
Q2.Write a Python program to implement Mini-Max Algorithm.
```

In [ ]:
```
#slip13(Q1)
```

In [ ]:
```
#slip21
```

In [ ]:
```
Q1.Write a python program to remove punctuations from the given string
```

In [ ]:
```
#slip3(Q1)
```

In [ ]:
```
Q2.Write a Python program to implement Heuristic search algorithm.
```

In [6]:
```
from queue import PriorityQueue
def heuristic(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])
def a_star(start, goal, grid):
    open_set = PriorityQueue()
    open_set.put((0, start))
    came_from = {}
    g_score = {start: 0}
    while not open_set.empty():
        _, current = open_set.get()
        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            return path[::-1] + [goal]
        for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
            neighbor = (current[0]+dx, current[1]+dy)
            if 0 <= neighbor[0] < len(grid) and 0 <= neighbor[1] < len(grid[0]) and grid[neighbor[0]][neighbor[
                tentative_g = g_score[current] + 1
                if tentative_g < g_score.get(neighbor, float('inf')):
                    came_from[neighbor] = current
                    g_score[neighbor] = tentative_g
                    f = tentative_g + heuristic(neighbor, goal)
                    open_set.put((f, neighbor))
    return None
grid = [
    [0,0,0],
    [1,1,0],
    [0,0,0]]
start, goal = (0,0), (2,2)
path = a_star(start, goal, grid)
print("Path:", path if path else "No path found")
```

```
Path: [(0, 1), (0, 2), (1, 2), (2, 2), (2, 2)]
```

In [ ]:
```
#slip22
```

In [ ]:
```
Q1.Write a Program to Implement Alpha-Beta Pruning using Python
```

In [7]:
```
def alpha_beta(node, depth, alpha, beta, maximizingPlayer, tree):
    if depth == 0 or node not in tree:
        return tree.get(node, 0)
    if maximizingPlayer:
        maxEval = float('-inf')
        for child in tree[node]:
            eval = alpha_beta(child, depth-1, alpha, beta, False, tree)
            maxEval = max(maxEval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break
        return maxEval
    else:
        minEval = float('inf')
        for child in tree[node]:
            eval = alpha_beta(child, depth-1, alpha, beta, True, tree)
            minEval = min(minEval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break
        return minEval
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
```

```
    'C': ['F', 'G'],
    'D': 3,
    'E': 5,
    'F': 6,
    'G': 9}
value = alpha_beta('A', 2, float('-inf'), float('inf'), True, tree)
print("Optimal value:", value)
```

```
Optimal value: 6
```

In [ ]:
```
Q2.Write a Python program to implement Simple Chatbot
#slip10(Q2)
```

In [ ]:
```
#slip23
```

In [ ]:
```
Q1.Write a Program to Implement Tower of Hanoi using Python.
```

In [8]:
```
def tower_of_hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    tower_of_hanoi(n-1, source, auxiliary, target)
    print(f"Move disk {n} from {source} to {target}")
    tower_of_hanoi(n-1, auxiliary, target, source)
tower_of_hanoi(3, 'A', 'C', 'B')
```

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

In [ ]:
```
Q2. Write a Python program for the following Cryptarithmetic problems
SEND + MORE = MONEY
#slip10(Q1)
```

In [ ]:
```
#slip24
```

In [ ]:
```
Q1.Write a python program to implement list operations
#slip5(Q1)
```

In [ ]:
```
Q2.Write a Python program to implement heuristic search algorithm
#slip21(Q2)
```

In [ ]:
```
#slip25
```

In [ ]:
```
Q1.Write a program to find and display perfect numbers from list
```

In [9]:
```
def is_perfect(n):
    return sum(i for i in range(1, n) if n % i == 0) == n
nums = [6, 28, 12, 496, 50, 8128, 100]
perfect_nums = [n for n in nums if is_perfect(n)]
print("Perfect numbers:", perfect_nums)
```

```
Perfect numbers: [6, 28, 496, 8128]
```

In [ ]:
```
Q2.Write a Python program to solve 8- Queen puzzle problem.
#slip13(Q2)
```

In [ ]:
```
```